



Sunbeam Institute of Information Technology

Pune and Karad

Module – Real Time Operating System

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Non preemptible

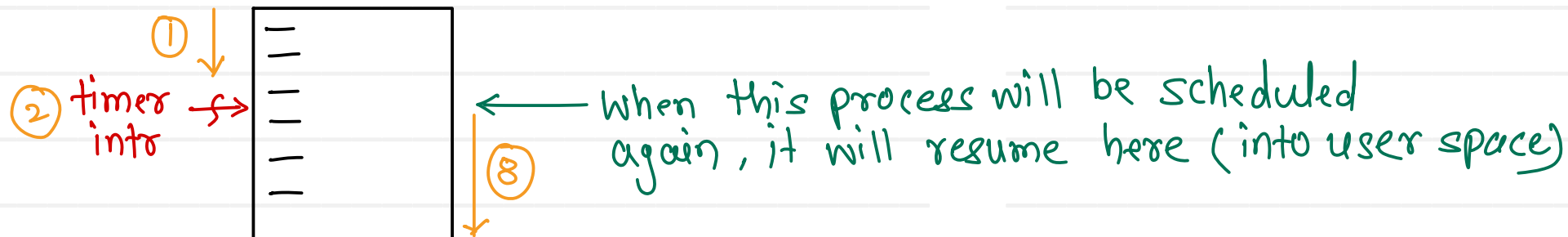
- it supports only user space preemption
- main focus is on throughput
- Server OS kernels

Preemptible

- it supports both user space & kernel space preemption
- main focus is on responsiveness
- RTOS kernel

User space preemption

Process



interrupt_handler() {

- ③ i. save execution context of current process
- ii. find address of ISR from IVT
- iii. call ISR()
- ⑤ iv. pid = CPU_scheduler() ← preemption
- ⑦ v. CPU_dispatcher(pid)

ISR() {

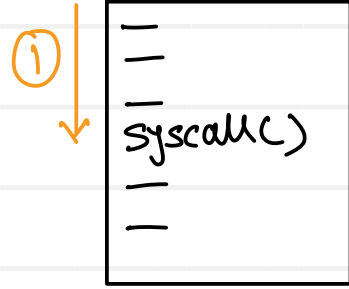
④ ↓

}

⑥

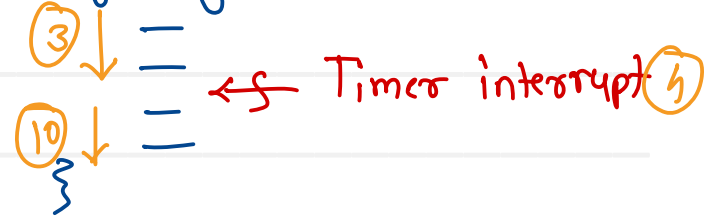
Userspace preemption

Process



When this process will be scheduled again, it will resume here (into user space)

sys_syscall() {



swi_handler() {

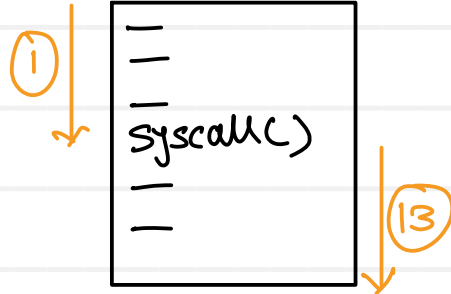
1. save execution context of current process
2. find address of actual syscall implementation from system call table
3. call actual syscall implementation
4. pid = CPU_scheduler(); ← preemption
5. CPU_Dispatcher(pid)

interrupt_handler() {

- i. save execution context of current process
- ii. find address of ISR from IVT
- iii. call ISR()
- iv. pid = CPU_scheduler() ← preemption is deferred
- v. CPU_Dispatcher(pid)

Kernel space preemption

Process



swi_handler() {

1. save execution context of current process
2. find address of actual syscall implementation from system call table
3. call actual syscall implementation
4. pid = CPU_scheduler();
5. CPU_Dispatcher(pid)

}

When process will be re scheduled, it will resume here (into kernel space)

sys_syscall() {

3 ↓

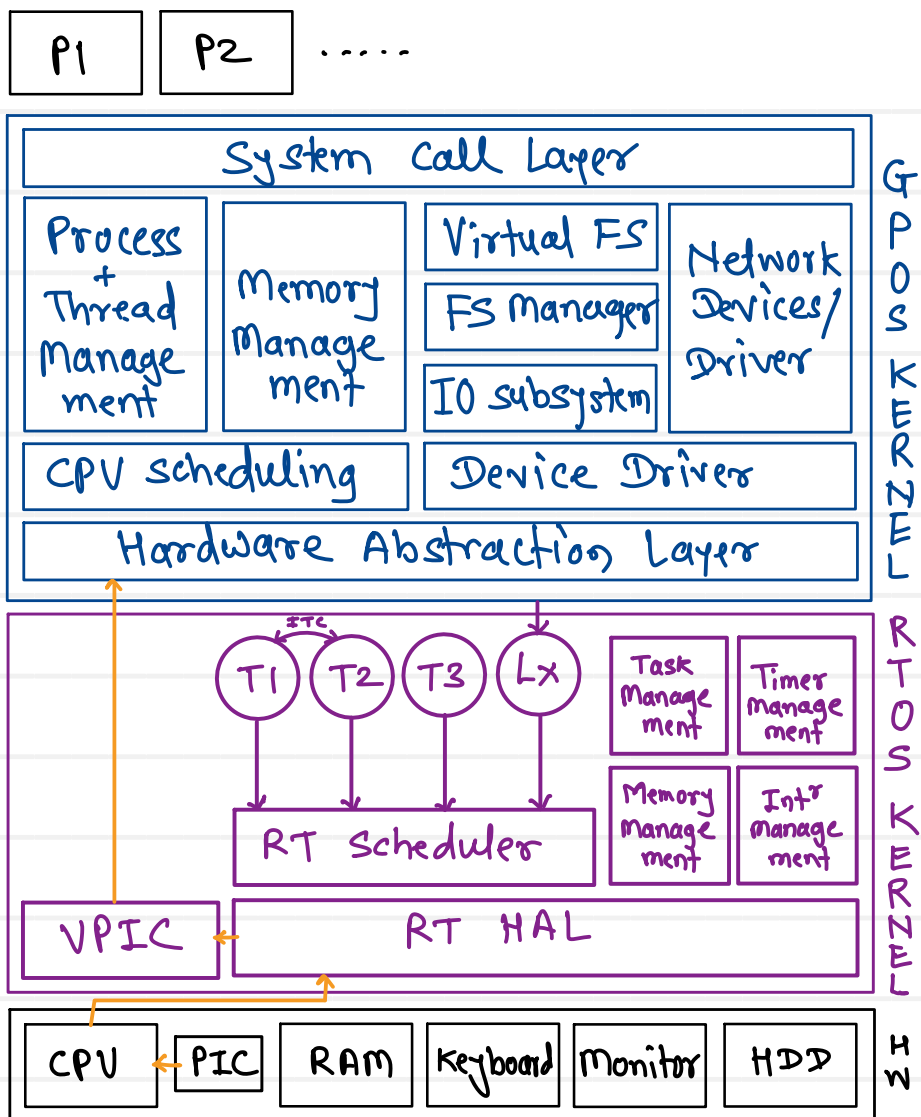
← Timer interrupt 4

10 ↓

interrupt_handler() {

- i. save execution context of current process
 - 5 ii. find address of ISR from IVT
 - iii. call ISR()
 - 7 iv. pid = CPU_scheduler() ← preemption
 - 9 v. CPU_Dispatcher(pid)
- 2
- ISR() {
- 6 ↓
- }

Dual kernel approach



- GPOS & RTOS kernels work together
- interrupts are given to RTOS kernel, if not handled then only passed to GPOS kernel.
- RTOS kernel works like virtual PIC for GPOS kernel.
- GPOS kernel can not manage any H/W interrupt directly.
- Linux kernel is treated as low priority task in RTOS.
- Whenever real time tasks are not available, linux kernel task get scheduled by RTOS kernel.

e.g. Xenomai, RTAI

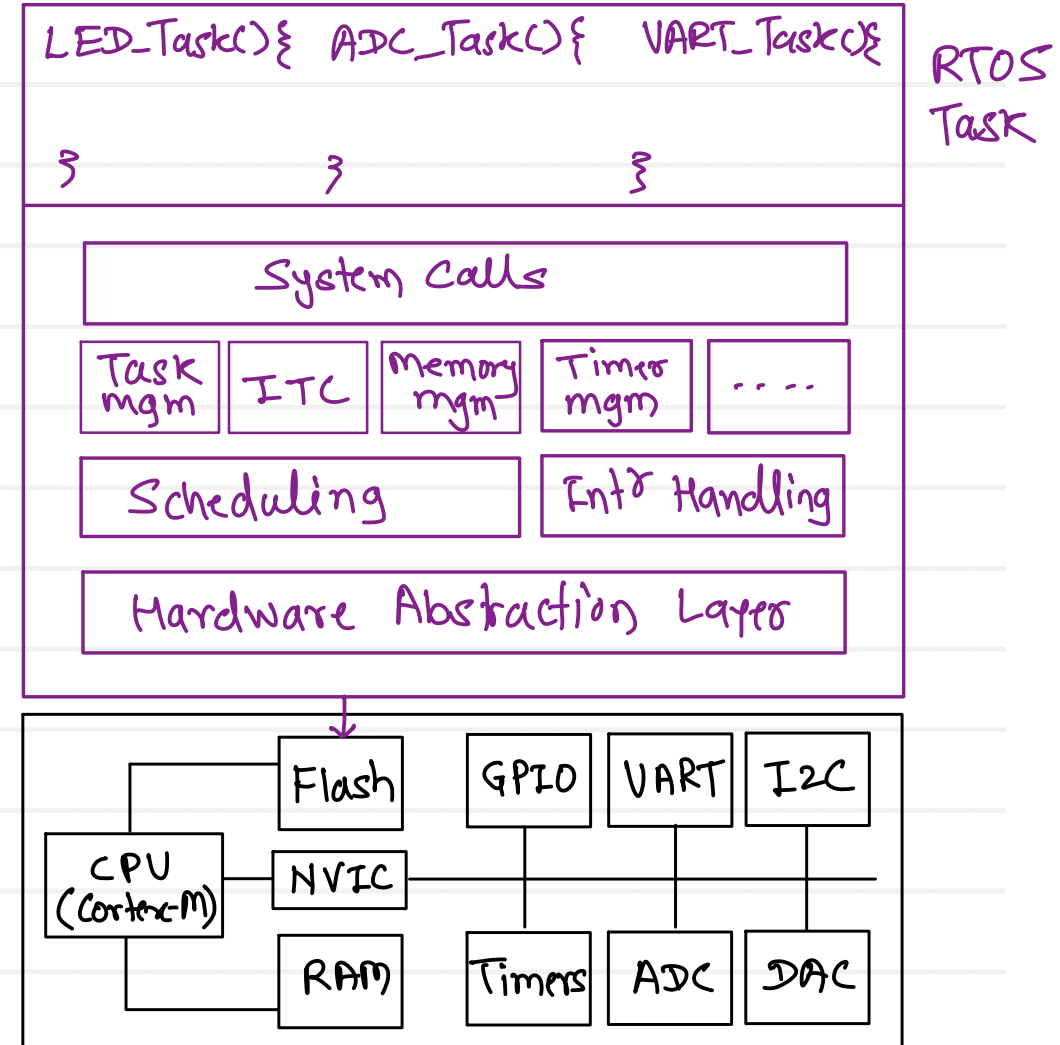
1. read value of temp
2. if temp goes beyond threshold, beep buzzer
3. read value of intensity
4. depending on intensity, adjust LED intensity
5. print status on UART
6. Use timers to measure timing

Advantages :

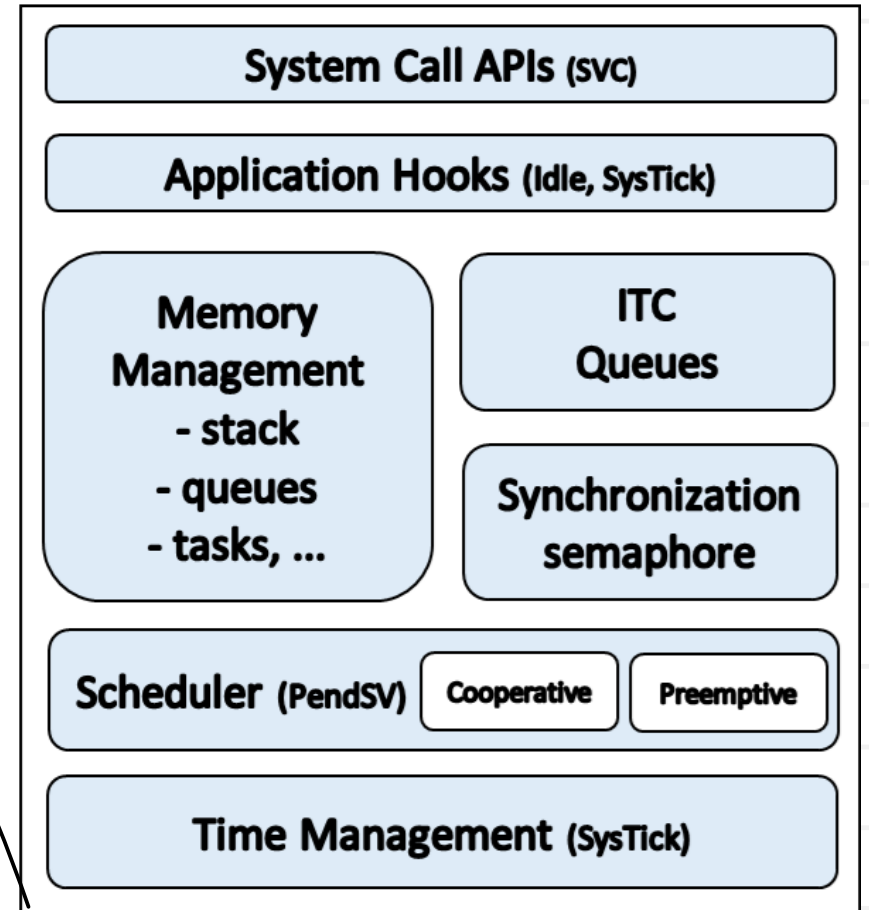
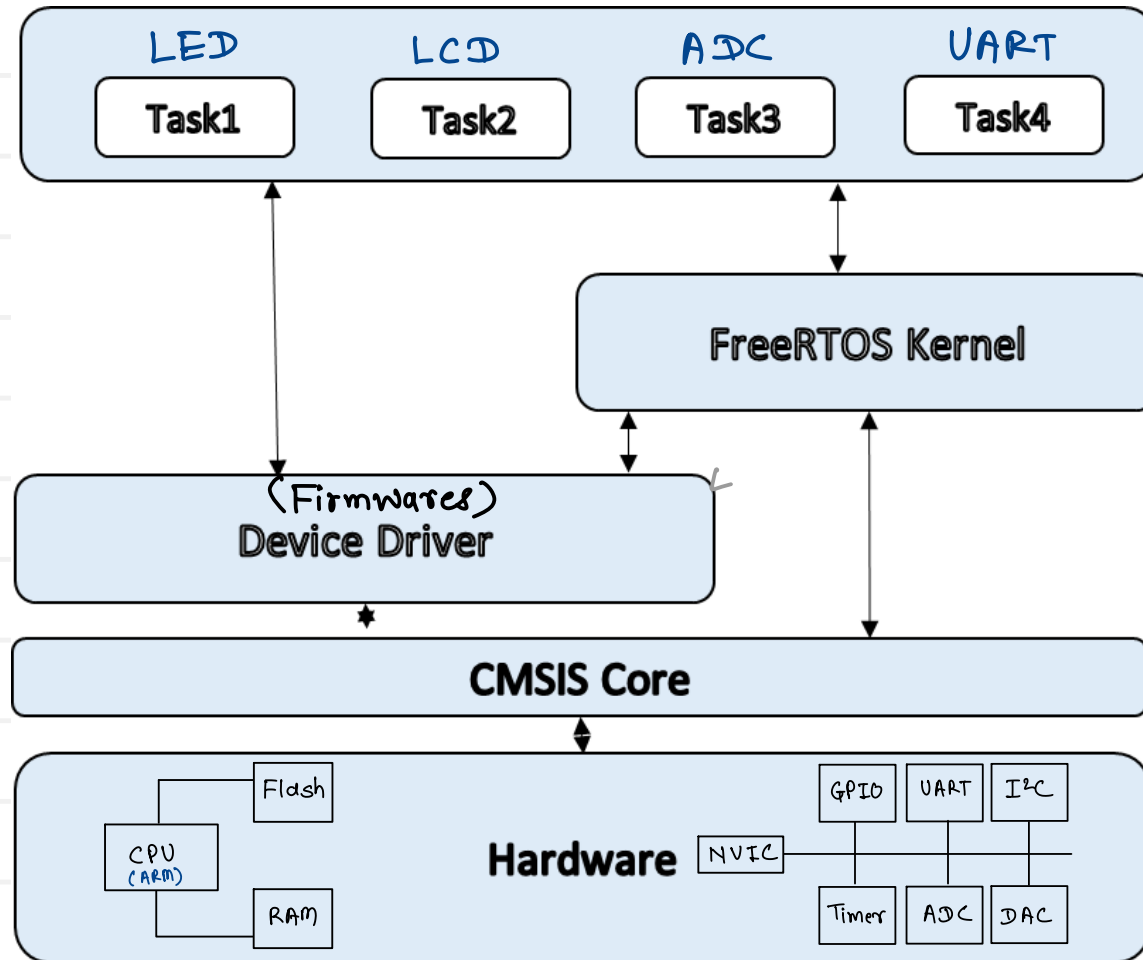
1. team work
2. readable
3. reusability
4. maintainance
5. better time mngm
6. portable
7. multitasking & commⁿ
8. Hardware abstractions

FreeRTOS, uCOS/II, QNX

Embedded
RTOS
Kernel



FreeRTOS Architecture





Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com